

- 副作用 != 负作用 : 这里的 副 是 其他 的意思,表示除了原有逻辑还会产生其他作用如:

- 发送网络请求
- 修改DOM
- 存储数据到浏览器
- 启动定时器

- `useEffect()` 组件渲染之后进行 (函数)

```
. // 执行顺序:  
: // 1. 组件函数执行, 返回 JSX  
: // 2. React 把 JSX 渲染到页面上 (此时 DOM 已存在)  
: // 3. useEffect 执行 (这时才能安全操作 DOM)  
:
```

因为会之后进行 dom 操作

`useEffect(, [])` 监听空数组,页面加载时执行

useRef 和 useState 的主要区别是什么?

选项:

- A. useRef 不能存储数据
- B. 修改 useRef 的值不会触发页面重新渲染

B

题目 11: useRef 使用场景

问题： 以下哪个最适合用 useRef?

选项：

- A. 存储要在页面上显示的数字
- B. 存储定时器的 ID

B

yaml

· 答案：

B

详细解析：

```
1 // useMemo: 缓存计算结果
2 const result = useMemo(() => computeExpensiveValue(a, b), [a, b]);
3 // result 是 computeExpensiveValue 的返回值
4
5 // useCallback: 缓存函数本身
6 const handleClick = useCallback(() => {
7   doSomething(a, b);
8 }, [a, b]);
9 // handleClick 是一个函数
```

useCallback = useMemo 的特殊版本，专门缓存函数。

使用场景：

- 把函数传给 memo 子组件
- 避免子组件不必要的重渲染

题目 18: Context 解决的问题

问题: Context 主要解决什么问题?

选项:

- A. 让组件渲染更快
- B. 避免 props 层层传递 (钻洞问题)
- C. 替代 useState
- D. 替代 useEffect

答案: B

详细解析:

Props 层层传递问题:

1 祖父(数据) → 父(不需要但必须传) → 子(需要数据)

Context 解决方案:

```
1 // 提供数据 (在顶层)
2 <MyContext.Provider value={数据}>
3   <App />
4 </MyContext.Provider>
```

题目 21: 获取路由参数

问题: 如何获取 URL `/user/123` 中的 id?

选项:

- A. `useRoute()`
- B. `useParams()`
- C. `useSearchParams()`
- D. `props.match.params.id`

答案:

B

详细解析:

```
1 // 路由配置
2 <Route path="/user/:id" element={<User />} />
3
4 // 在 User 组件中获取
5 import { useParams } from 'react-router-dom';
6
7 function User() {
8   const { id } = useParams(); // id = "123"
9   return <div>用户ID: {id}</div>;
```

```
4     _____(timer);  
5   };  
6 }, []);
```

· 答案:

`clearInterval`

解析: 返回清理函数, 在组件卸载时执行清理工作。

填空 4

补全: 创建 useRef

```
1 const inputRef = _____(null);
```

· 答案:

填空 5

补全: 绑定 ref 到元素

```
1 <input _____={inputRef} type="text" />
```

· 答案:

```
useEffect(() => { setCount(count + 1); }, []); // 依赖项数组为空
```

改错 3: useRef 错误修改方式

错误代码:

```
1 function Example() {
2   const countRef = useRef(0);
3
4   const add = () => {
5     countRef = countRef + 1;
6   };
7
8   return <button onClick={add}>+1</button>;
9 }
```

修正方法一: 使用函数式更新 (推荐)

```
1 useEffect(() => {
2   const timer = setInterval(() => {
3     setCount(c => c + 1); // 函数式更新不依赖外部 count
4   }, 1000);
5
6   return () => clearInterval(timer);
7 }, []); // 空数组, 只在挂载时设置定时器
```

修正方法二: 添加终止条件

```
1 useEffect(() => {
2   if (count >= 10) return; // 终止条件
3
4   const timer = setTimeout(() => {
5     setCount(count + 1);
6   }, 1000);
7
8   return () => clearTimeout(timer);
9 }, [count]);
```

工作原理：

1. 依赖项数组 `[count]`：我们告诉 React：“这个 effect 的执行依赖于 `count` 的值。当 `count` 变化时，请重新运行这个 effect。”
2. `if (count >= 10) return;`：在 effect 开始时，我们检查一个条件。如果条件满足（例如 `count` 已经达到了 10），我们就 `return`，不执行后续的 `setTimeout`。这可以防止在达到某个状态后继续创建新的定时器。
3. 闭包问题依然存在：虽然这个方法可以防止无限循环，但它并没有从根本上解决 `count` 是“陈旧状态”的问题。如果 `count` 的值变化很快，`setTimeout` 内部的 `count + 1` 仍然可能使用的是旧的 `count` 值。不过，在很多场景下，这种“轻微的不准确”是可以接受的，特别是当 effect 的主要目的是在某个条件下停止时。

适用场景：当你需要在 effect 中设置一个终止条件，并且对状态的实时性要求不高时，可以使用此方法。

```
useEffect(() => {  
  if (count >= 10) return; // 终止条件  
  
  const timer = setTimeout(() => {  
    setCount(count + 1);  
  }, 1000);  
  
  return () => clearTimeout(timer);  
}, [count]); // 依赖项是 [count]
```

```
useEffect(() => {  
  if (count >= 10) return; // 终止条件  
  
  const timer = setTimeout(() => {  
    setCount(count + 1);  
  }, 1000);  
  
  return () => clearTimeout(timer);  
}, [count]); // 依赖项是 [count]  
// 这个是在每次count变化时都产生一个计数器1s后+1
```